

IOT 2025–26

DEFAULT PROJECT : Smarter Bicycle Mobility in Genoa

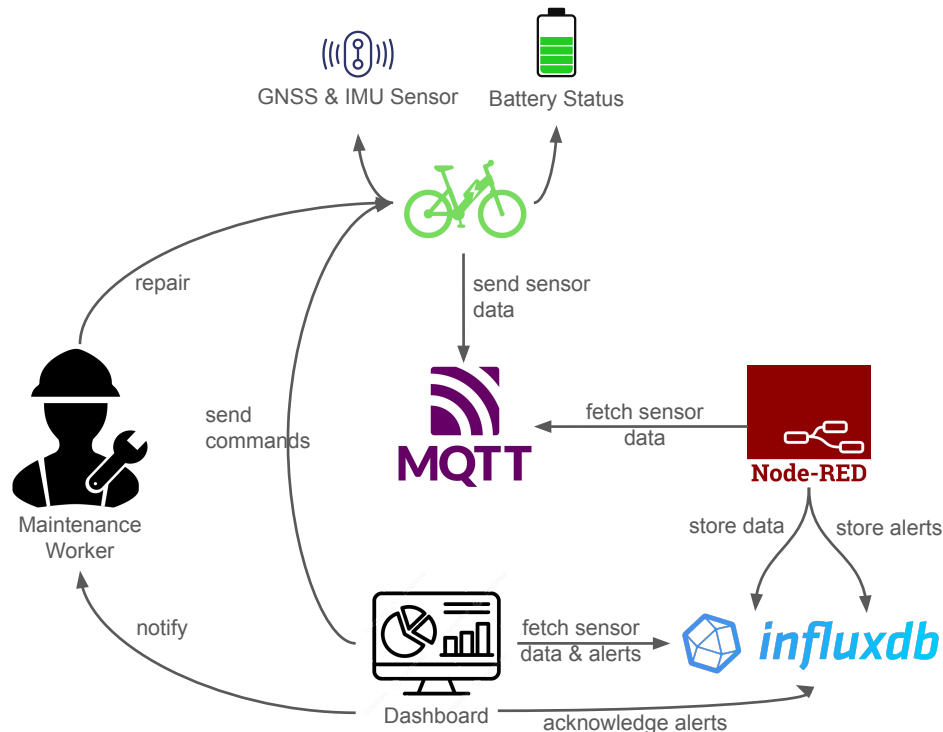
INTERNET OF THINGS - 90524

**Yonatan Sisay Teshome 8529864
Henri Junior Houphouet 7505368
Simon Hanselmann 9267578**

Architecture of the System

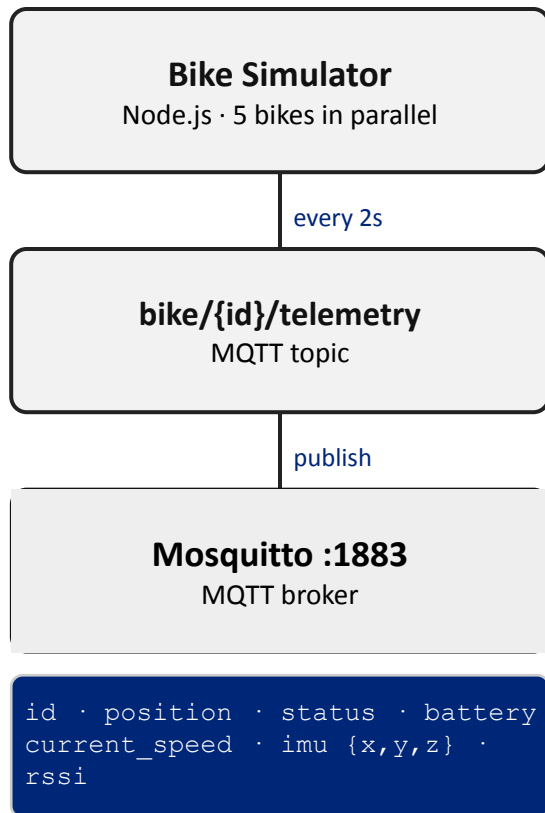
- Design goals:

- Simple bike hardware
- Reliable and scalable communication
- Quick detection of issues
- Efficient data querying
- Simple to use dashboard with fleet overview and important metrics on first glance
- Quick and reliable interaction with bike and workers



Bike Simulator & Data Acquisition

Edge device simulation with Node.js



THE KEY FEATURES

Sensor simulation

GPS position · IMU accelerations (x, y, z) · battery drain · RSSI signal
All modeled with Gaussian noise for realistic variation per zone

14 real Genoa routes

Routes fetched live from OSRM : Porto Antico, Brignole, Boccadasse, Castelletto and more

Docker deployment

All 5 bikes run in parallel inside a single Docker container
Starts automatically with docker compose up -d

30 days historical data

Seeded automatically on first startup , ~600k data points in InfluxDB
Each bike has a different usage profile to simulate fleet diversity

Network Simulation

Realistic IoT packet loss and latency based on urban signal coverage in Genoa

RSSI ZONES

Zone	RSSI	Quality
Porto Antico	-58 dBm	Excellent
Brignole	-61 dBm	Good
San Martino	-67 dBm	Average
Sampierdarena	-72 dBm	Weak
Caruggi	-82 dBm	Poor

PACKET LOSS & LATENCY

RSSI	Drop prob.	Latency avg
≥ -65 dBm	1%	~50ms
≥ -75 dBm	5%	~120ms
≥ -85 dBm	25%	~300ms
≥ -95 dBm	60%	~800ms
< -95 dBm	90%	~800ms

HOW IT WORKS

1. RSSI computed per position

Each tick, the bike's GPS is checked against coverage zones. RSSI = Gaussian noise around zone mean.

2. Packet drop (BikeSimulator.js)

Before sending, a random number is compared to a drop probability derived from the RSSI , if below, the packet is lost.

3. Latency simulation (main.js)

Surviving packets are delayed via `setTimeout()` with Gaussian delay based on current RSSI.

Scenarios & Historical Data

Injectable test scenarios and 30 days of seeded data for dashboard demo

SIMULATION SCENARIOS

normal

default

Bikes ride 14 Genoa routes continuously.
Parking with exponential distribution, ~20 min avg.
Command: `node main.js normal bike-ge-001`

fall

high severity

At tick 10, IMU shifts permanently: $z \sim 1.1 \text{ m/s}^2$, speed=0.
Triggers fall detection alert in Node-RED Flow B.
Command: `node main.js fall bike-ge-001`

low_battery

low severity

Battery forced to 15% at tick 1.
Immediately triggers low battery alert.
Command: `node main.js low_battery bike-ge-001`

HISTORICAL DATA — 30 DAYS

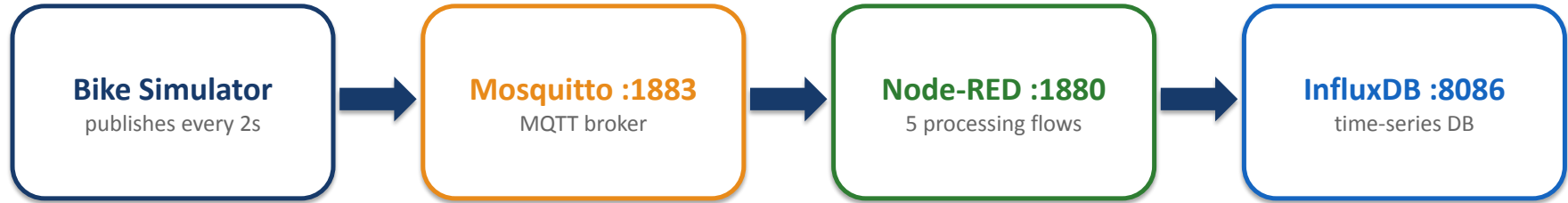
~600,000 data points · 5 bikes · 30 days

Seeded on Docker startup no manual steps needed

BIKE USAGE PROFILES

Bike	Mean parking	Usage
bike-ge-001	15 min	Very active
bike-ge-002	30 min	Normal
bike-ge-003	60 min	Low usage
bike-ge-004	20 min	Active
bike-ge-005	90 min	Rarely used

Data Processing Pipeline



MQTT Broker

Mosquitto receives telemetry on bike/{id}/telemetry

Stream Processing

5 Node-RED flows: ingest, fall detection, parking, battery, connectivity

Time-Series Storage

InfluxDB stores bike telemetry + alerts in bike_data bucket

Node-RED Processing Flows

Flow A Ingest & Store

Subscribes to
bike+/telemetry

Writes bike
measurement to
InfluxDB

Tracks bikeLastSeen
timestamp

Flow B Fall Detection

Triggers when $|z - 9.8|$
 $> 4.0 \text{ m/s}^2$

or total acceleration $>$
 25 m/s^2

Only fires when bike is
rented

Flow C Parking Violation

Fires when a locked
bike is outside

all authorized parking
zones

Uses
shared/parking-zones.j
son

Flow D Battery Alerts

Tiered severity:

$\leq 15\%$ (low) · $\leq 5\%$
(med) · 0% (high)

Fires on severity tier
change

Flow E Connectivity

Checks every 15s for
silent bikes

Alerts if no telemetry
for $> 60\text{s}$

Auto-clears when bike
reconnects

Alert System

Type	Severity	Trigger Condition
fall	high	$ z - 9.8 > 4.0$ OR total accel $> 25 \text{ m/s}^2$ (rented only)
parking_violation	medium	Locked bike outside all authorized parking zones
low_battery	low / med / high	$\leq 15\%$ (low), $\leq 5\%$ (medium), 0% (high)
connectivity	medium	No telemetry for $> 60\text{s}$; auto-clears on reconnect

Alert Acknowledgment

- Alerts are append-only events
- Ack written as separate alert_ack measurement
- Tags: bike_id, alert_id
- Fields: acked, source

InfluxDB Schema

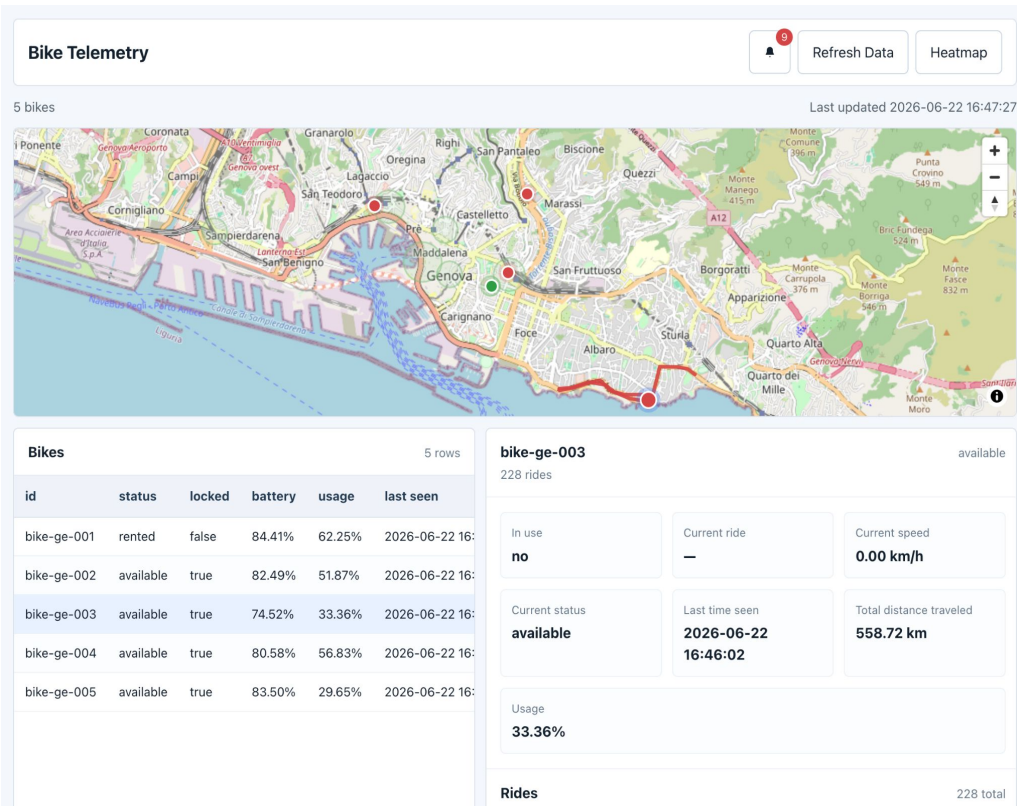
INFLUXDB SCHEMA

Bucket: bike_data - Org: iot-bikes

Measurement	Tags	Key Fields
bike	id	lat, lng, speed, imu_x/y/z, battery, locked, status
alert	bike_id, type, severity	alert_id, message
alert_ack	bike_id, alert_id	acked, source

Dashboard

- Implemented with React
- Fetches data from InfluxDB
- Current locations of bikes
- Detailed view for each bike
- Individual ride visualization
- Alert notifications and management



Heatmap

- For rides and parking spots in the last 24 hours or all time
- Detect popular routes around the city
- Find areas where bikes are parking for a long time

